

EGE353

Autonomous Mobile Robotics

Service mobile robots, concepts of path planning, navigation, obstacle avoidance, simultaneous localisation and mapping.

Lab 6: Create ros launch files

Chan Kit Wai
Tel: 65500559
Chan_kit_wai@nyp.edu.sg



Synopsis

This module aims to equip learners with the knowledge of service robots and their applications. Learners will also study the concepts of path planning, navigation, obstacle avoidance, simultaneous localisation and mapping. They will be able to implement mobile robotics system to carry out tasks autonomously.

Module Learning Outcomes

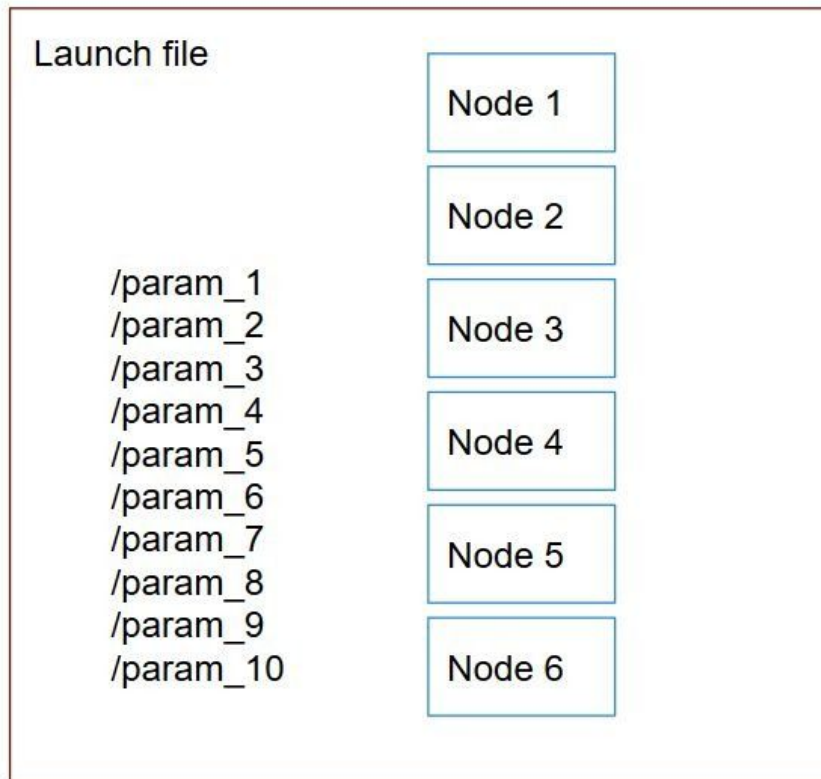
At the end of the module, learners are able to:

- Explain the features of different sensors and actuators used in autonomous mobile robotics.
- Perform the analysis of laser and vision sensor data using visualization tools.
- Implement mobile robotics system using methods such as mapping, localization, path planning, obstacle avoidance and motion control.

ROS launch files

In most of the introductory tutorials, you have been opening new terminals for every new node you run. As you create more complex systems with more and more nodes running simultaneously, opening terminals and reentering configuration details becomes tedious.

Launch files allow you to start up and configure a number of executables containing ROS 2 nodes simultaneously. Running a single launch file with the **ros2 launch** command will start up your entire system - all nodes and their configurations - at once.



ROS launch files

Let's start by seeing how roslaunch enables us to start many nodes at once. The basic idea is to list, in a specific file format, a group of nodes that should be started at the same time.

The launch system in ROS 2 is responsible for helping the user describe the configuration of their system and then execute it as described. The configuration of the system includes what programs to run, where to run them, what arguments to pass them, and ROS specific conventions which make it easy to reuse components throughout the system by giving them each different configurations. It is also responsible for monitoring the state of the processes launched, and reporting and/or reacting to changes in the state of those processes.

Terminal 1

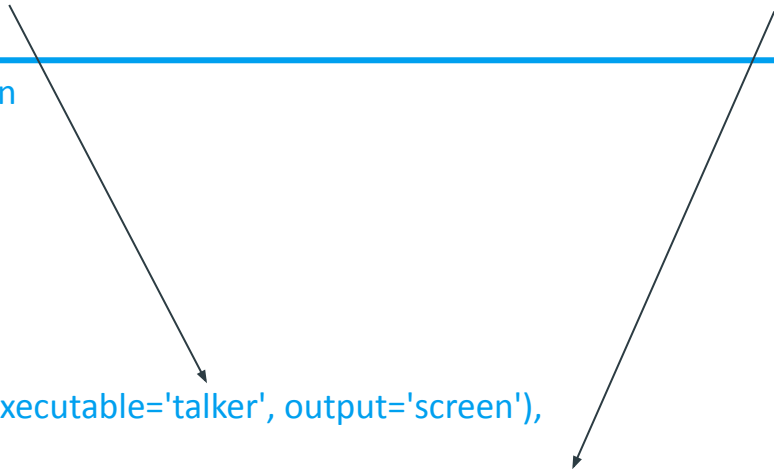
```
~/dev_ws$ ros2 run cpp_pubsub talker
```

Terminal 2

```
~/dev_ws$ ros2 run cpp_pubsub listener
```

```
from launch import LaunchDescription
import launch_ros.actions
```

```
def generate_launch_description():
    return LaunchDescription([
        launch_ros.actions.Node(
            package='demo_nodes_cpp', executable='talker', output='screen'),
        launch_ros.actions.Node(
            package='demo_nodes_cpp', executable='listener', output='screen'),
    ])
```

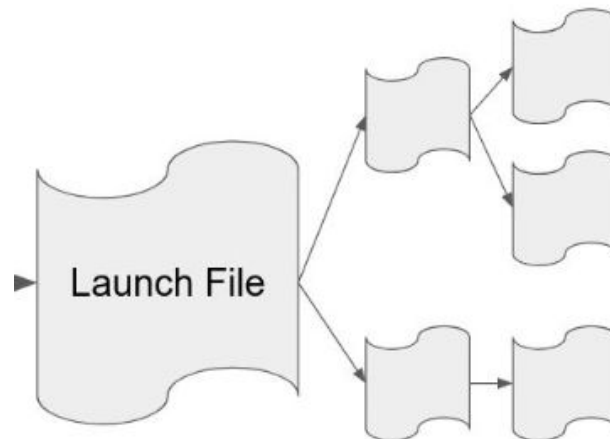


ROS launch files

Launch files written in Python can start and stop different nodes as well as trigger and act on various events. The package providing this framework is `launch_ros`, which uses the non-ROS-specific launch framework underneath. The launch file below is written in Python, but you can also use XML and YAML to create launch files. You can see a comparison of these different ROS 2 launch formats in [Using Python, XML, and YAML for ROS 2 Launch Files](#).

- Setup command line arguments with defaults
- Include another launch file
- Include another launch file in another namespace
- Start a node and setting its namespace
- Start a node, setting its namespace, and setting parameters in that node (using the args)
- Create a node to remap messages from one topic to another

Python, XML, and YAML for ROS 2 Launch Files.



ROS launch files

A ROS 2 launch file is a python script that interfaces with the ROS 2 *launch* and *launch_ros* libraries.

- Note the **include's** at the top of the *demo.launch.py* file.

A launch file requires a **def generate_launch_description()** function to be defined. It needs to return a *LaunchDescription* object.

A **LaunchDescription** holds an array of *LaunchDescriptionEntity*'s (ie 'launch to-do items'). Each *LaunchDescriptionEntity* will be performed upon the *ros2 launch ...* CLI command call.

For our *demo.launch.py* launch file, the **LaunchDescription** is created with an array of 2 *Node LaunchDescriptionEntity*'s:

- Launch another ROS 2 program **ros2 run cpp_pubsub talker** from the **cpp_pubsub** package.



Q&A

Q1)

What is a ros launch file ?



ROS launch files

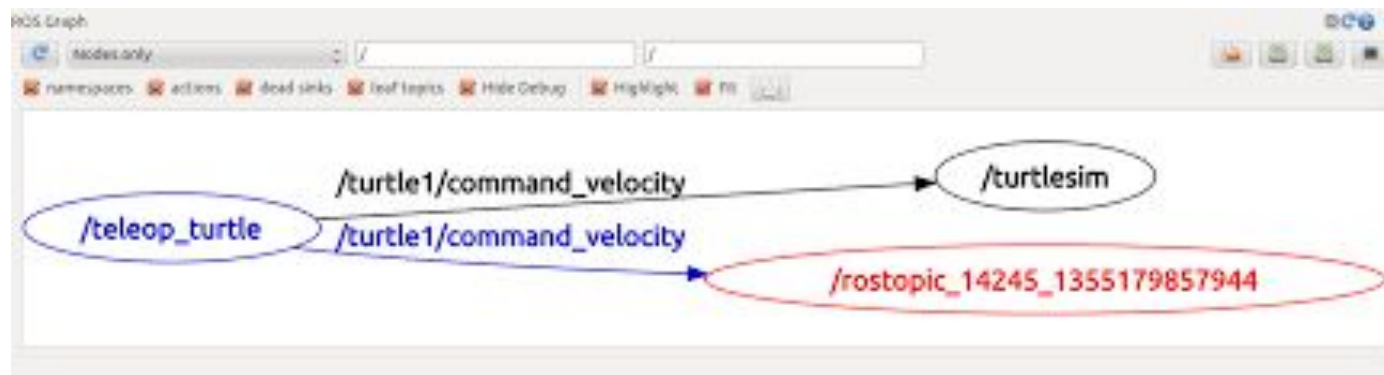
- **Rename node**

You may want to rename your node, for example if you want to start 2 different nodes from the same executable.

...

```
talker_node = Node(  
    package="demo_nodes_cpp",  
    executable="talker",  
    name="my_talker"  
)
```

...

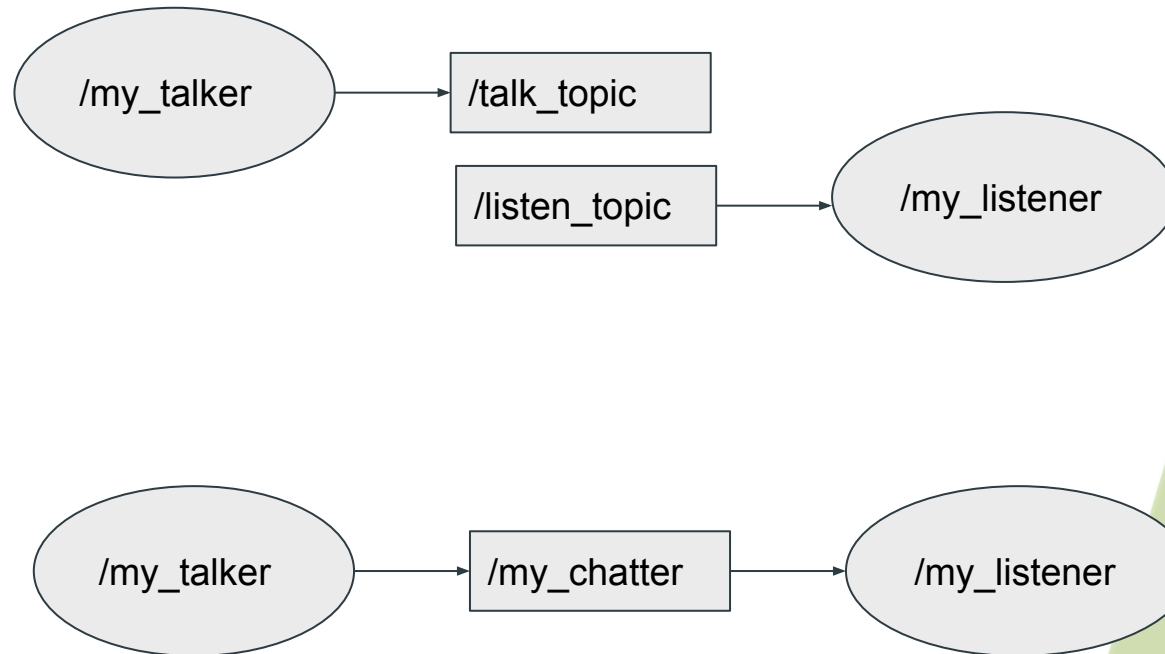


ROS launch files

- **Topic/Service remapping**

To remap a topic/service inside a node, add a `remappings[]` array inside the `Node` object, and add a tuple: first value is the current name, second value is the new name. i.e. `remappings=[ ("old_topic", "new_topic") ]`

```
...
talker_node = Node(
    package="demo_nodes_cpp",
    executable="talker",
    name="my_talker",
    remappings=[
        ("talk_topic", "my_chatter")
    ]
)
listener_node = Node(
    package="demo_nodes_py",
    executable="listener",
    name="my_listener",
    remappings=[
        ("listen_topic", "my_chatter")
    ]
)
```



Q&A

Q2)

Why do we need remapping ?

- 1) So that two nodes can communicate using the same topic
- 2) So that two topics can communicate using the same node
- 3) So that it can map out the room
- 4) So that the old topic get a new topic name.



Task 1: Create a simple launch file

Task 1: Create a simple launch file

Open Terminal 1

Create a new directory. For this example, it will be launch_ws. For other workspace it can be for example sensor_ws or robotdog_ws. Ensure you are in the correct directory. Note that the 2 files pub.cpp and sub.cpp are cloned to local directory.

```
user:~$ mkdir launch_ws
user:~$ cd launch_ws
user:~/launch_ws$ mkdir src
user:~/launch_ws$ cd src
user:~/launch_ws/src$ git clone https://github.com/Adlink-ROS/ros2\_simple\_example.git
Cloning into 'ros2_simple_example'...
remote: Enumerating objects: 32, done.
remote: Counting objects: 100% (32/32), done.
remote: Compressing objects: 100% (25/25), done.
remote: Total 32 (delta 4), reused 28 (delta 3), pack-reused 0
Unpacking objects: 100% (32/32), 6.56 KiB | 1.09 MiB/s, done.
user:~/launch_ws/src$ ls
ros2_simple_example
user:~/launch_ws/src$ cd ros2_simple_example/
user:~/launch_ws/src/ros2_simple_example$ ls
README.md  cpp_pubsub  examples.repos  py_pubsub
user:~/launch_ws/src/ros2_simple_example$ cd cpp_pubsub/src
user:~/launch_ws/src/ros2_simple_example/cpp_pubsub/src$ ls
pub.cpp  sub.cpp
user:~/launch_ws/src/ros2_simple_example/cpp_pubsub/src$ cd ~/launch_ws/
```

Task 1: Create a simple launch file

Terminal 1

Note that the `launch_ws` directory only have `src` folder before running the colon build. After colon build, 3 other directories are created.

```
user:~/launch_ws$ ls
src
user:~/launch_ws$ colcon build
Starting >>> cpp_pubsub
Starting >>> py_pubsub
Finished <<< cpp_pubsub [0.52s]
Finished <<< py_pubsub [0.81s]

Summary: 2 packages finished [0.96s]
user:~/launch_ws$ ls
build install log src
user:~/launch_ws$ source install/setup.bash
user:~/launch_ws$ ros2 run cpp_pubsub talker
[INFO] [1653554825.960436030] [minimal_publisher]: Publishing: 'Hello, world! 0'
[INFO] [1653554826.460414356] [minimal_publisher]: Publishing: 'Hello, world! 1'
```

Do not stop the program running.

Task 1: Create a simple launch file

Open Terminal 2

```
user:~$ cd launch_ws
user:~/launch_ws$ source install/setup.bash
user:~/launch_ws$ ros2 run cpp_pubsub listener
[INFO] [1653554867.458970508] [minimal_subscriber]: I heard: 'Hello, world! 83'
[INFO] [1653554867.958806265] [minimal_subscriber]: I heard: 'Hello, world! 84'
[INFO] [1653554868.458935353] [minimal_subscriber]: I heard: 'Hello, world! 85'
[INFO] [1653554868.958785895] [minimal_subscriber]: I heard: 'Hello, world! 86'
[INFO] [1653554869.458972529] [minimal_subscriber]: I heard: 'Hello, world! 87'
[INFO] [1653554869.958773254] [minimal_subscriber]: I heard: 'Hello, world! 88'
[INFO] [1653554870.458768358] [minimal_subscriber]: I heard: 'Hello, world! 89'
[INFO] [1653554870.958679137] [minimal_subscriber]: I heard: 'Hello, world! 90'
[INFO] [1653554871.458721078] [minimal_subscriber]: I heard: 'Hello, world! 91'
^C[INFO] [1653554871.504916144] [rclcpp]: signal_handler(signal_value=2)
user:~/launch_ws$
```

CTR+C to stop the program running in Terminal 2.

CTR+C to stop the program running in Terminal 1.

Task 1: Create a simple launch file

Open Terminal 3

In this terminal, you will be creating a launch file to run the talker and listener node in a single file. Use the touch command to create a new python launch file as follows.

```
user:~$ cd launch_ws/src/ros2_simple_example/cpp_pubsub/  
user:~/launch_ws/src/ros2_simple_example/cpp_pubsub$ mkdir launch  
user:~/launch_ws/src/ros2_simple_example/cpp_pubsub$ cd launch  
user:~/launch_ws/src/ros2_simple_example/cpp_pubsub/launch$ touch talk_listen.launch.py  
user:~/launch_ws/src/ros2_simple_example/cpp_pubsub/launch$ ls  
talk_listen.launch.py  
user:~/launch_ws/src/ros2_simple_example/cpp_pubsub/launch$ cd ~/launch_ws  
user:~/launch_ws
```

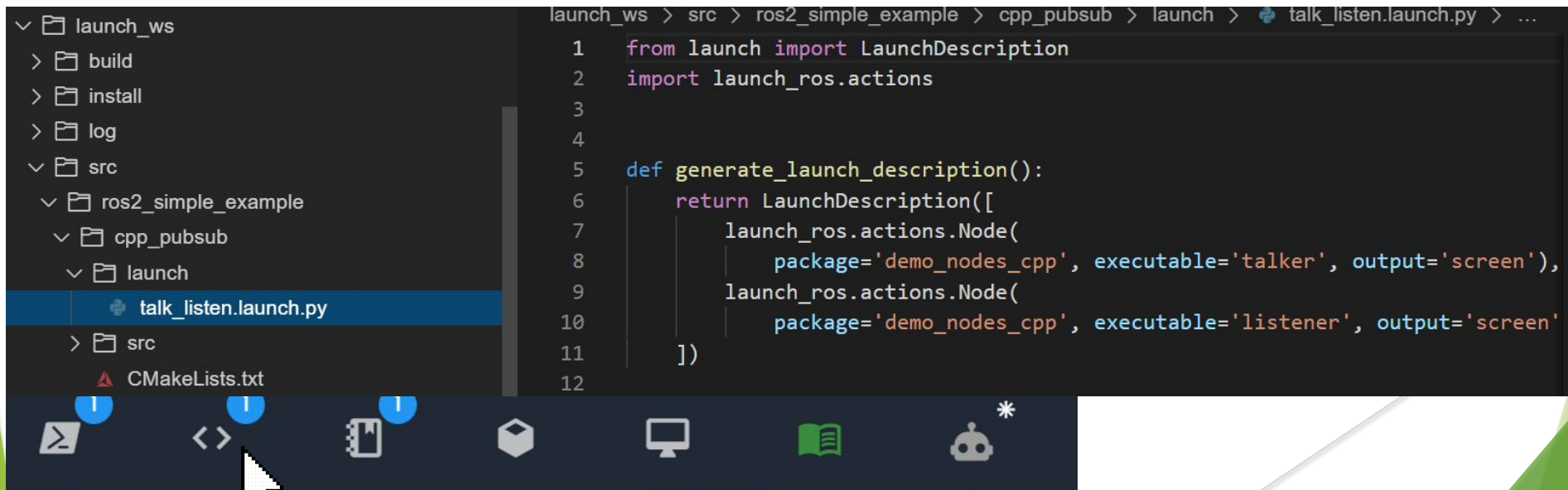
Touch is to create the file, copy and paste the content of the launch file in the next page.

Task 1: Create a simple launch file

Open the code editor, copy and paste the code. Save the file. These launch files are also available in polite mall.

```
from launch import LaunchDescription
import launch_ros.actions

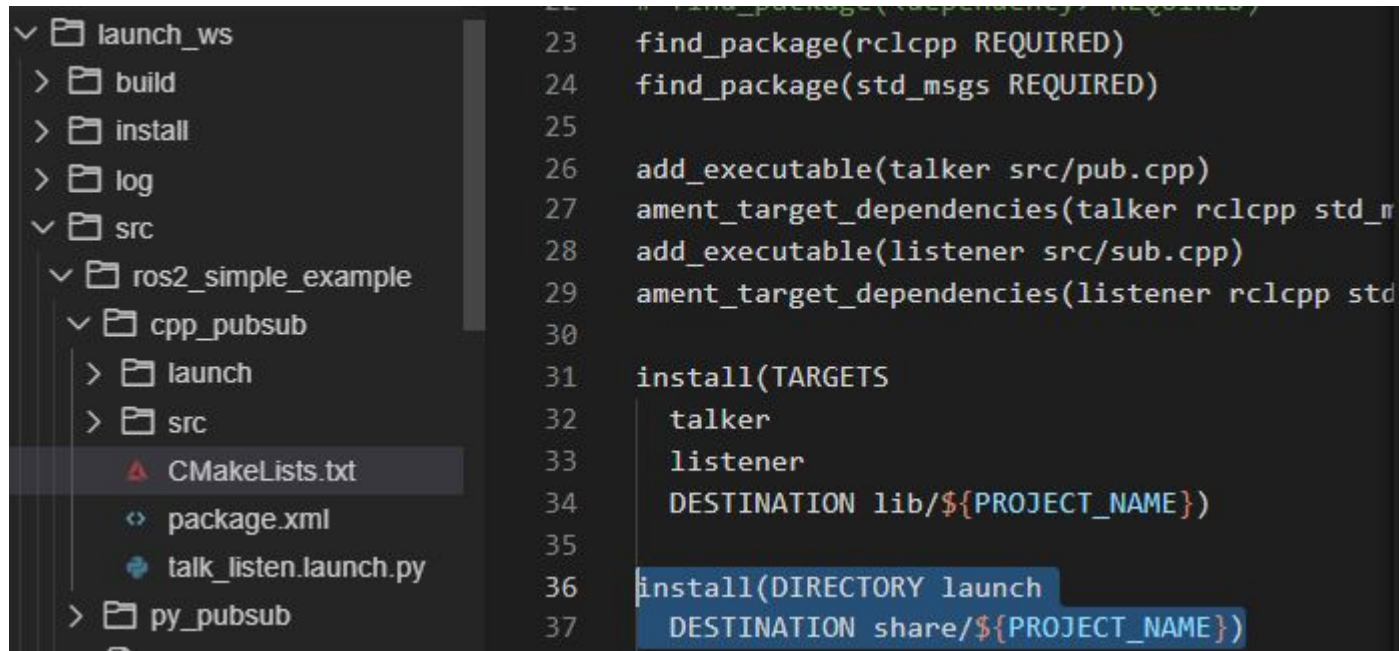
def generate_launch_description():
    return LaunchDescription([
        launch_ros.actions.Node(
            package='demo_nodes_cpp', executable='talker', output='screen'),
        launch_ros.actions.Node(
            package='demo_nodes_cpp', executable='listener', output='screen'),
    ])
```



Task 1: Create a simple launch file

Now, we need to tell ROS to recognize the launch file. Open up CMakeLists.txt in the IDE/ Code editor and add the following lines at the bottom of it. Copy and paste the following code to line 36

```
install(DIRECTORY launch
        DESTINATION share/${PROJECT_NAME})
```



```
23 find_package(rclcpp REQUIRED)
24 find_package(std_msgs REQUIRED)
25
26 add_executable(talker src/pub.cpp)
27 ament_target_dependencies(talker rclcpp std_msgs)
28 add_executable(listener src/sub.cpp)
29 ament_target_dependencies(listener rclcpp std_msgs)
30
31 install(TARGETS
32     talker
33     listener
34     DESTINATION lib/${PROJECT_NAME})
35
36 install(DIRECTORY launch
37     DESTINATION share/${PROJECT_NAME})
```

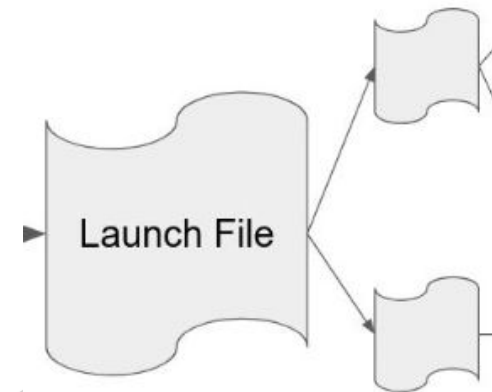
Task 1: Create a simple launch file

Terminal 3

```
user:~/launch_ws$ colcon build
Starting >>> cpp_pubsub
Starting >>> py_pubsub
Finished <<< cpp_pubsub [0.51s]
Finished <<< py_pubsub [0.80s]

Summary: 2 packages finished [0.94s]
user:~/launch_ws$ ls
build install log src
user:~/launch_ws$ source install/setup.bash
user:~/launch_ws$ ros2 launch cpp_pubsub talk_listen.launch.py
[INFO] [launch]: All log files can be found below
/home/user/.ros/log/2022-05-26-10-14-01-074370-4_xterm-1663
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [talker-1]: process started with pid [1665]
[INFO] [listener-2]: process started with pid [1667]
[talker-1] [INFO] [1653560042.177291472] [talker]: Publishing: 'Hello World: 1'
[listener-2] [INFO] [1653560042.177670283] [listener]: I heard: [Hello World: 1]
[talker-1] [INFO] [1653560043.177227271] [talker]: Publishing: 'Hello World: 2'
[listener-2] [INFO] [1653560043.177484956] [listener]: I heard: [Hello World: 2]
```

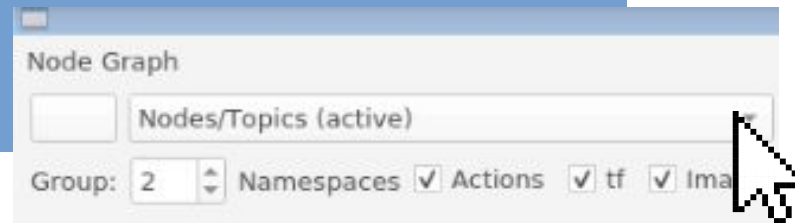
Both talker and listener nodes are launch by the talk_listen.launch.py



Task 1: Create a simple launch file

Terminal 4

```
user:~$ ros2 topic list  
/chatter  
/parameter_events  
/rosout  
user:~$ ros2 node list  
/listener  
/talker  
user:~$ rqt_graph
```

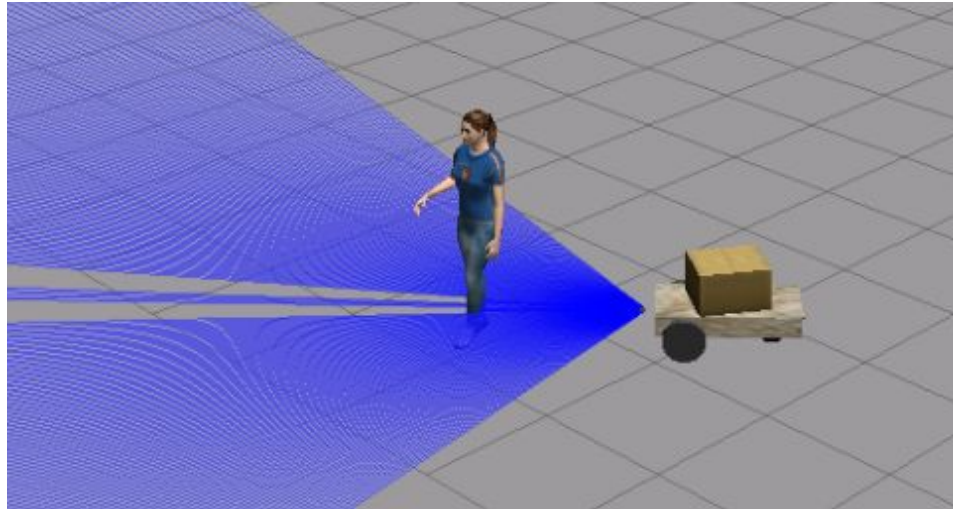


Stop all previous program Ctr+C and close all terminal.

What is the name of the topic ?



Task 2: Create a follow me robot launch file



Task 2: Follow me robot

Open a new Terminal 1

```
launch_ws$ cd src
user:~/launch_ws/src$ git clone https://github.com/1at7/teleop_cpp_ros2.git
Cloning into 'teleop_cpp_ros2'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 18 (delta 4), reused 9 (delta 1), pack-reused 0
Unpacking objects: 100% (18/18), 18.33 KiB | 3.05 MiB/s, done.
user:~/launch_ws/src$ ls
ros2_simple_example teleop_cpp_ros2
```

Note that the teleop_cpp_ros2 is cloned to the directory. It will be use as a keyboard controller to the dolly robot.

```
user:~/launch_ws/src$ git clone https://github.com/chapulina/dolly -b foxy
Cloning into 'dolly'...
remote: Enumerating objects: 496, done.
remote: Counting objects: 100% (111/111), done.
remote: Compressing objects: 100% (40/40), done.
remote: Total 496 (delta 77), reused 77 (delta 69), pack-reused 385
Receiving objects: 100% (496/496), 70.35 MiB | 11.75 MiB/s, done.
Resolving deltas: 100% (223/223), done.
user:~/launch_ws/src$ ls
dolly ros2_simple_example teleop_cpp_ros2
user:~/launch_ws/src$ cd ~/launch_ws
```

Task 2: Follow me robot

Terminal 1

```
user:~/launch_ws$ colcon build
Starting >>> dolly_follow
Starting >>> cpp_pubsub
Starting >>> py_pubsub
Starting >>> teleop_cpp_ros2
Finished <<< cpp_pubsub [1.21s]
Finished <<< py_pubsub [1.70s]
Finished <<< teleop_cpp_ros2 [9.11s]
Finished <<< dolly_follow [11.7s]
Starting >>> dolly_gazebo
Starting >>> dolly_ignition
[13.182s] WARNING:colcon.colcon_cmake.task.cmake.build:Could not run installation step for package
'dolly_ignition'because it has no 'install' target
--- stderr: dolly_ignition
CMake Warning at CMakeLists.txt:8 (message):
  Ignition Gazebo 3 not found, proceeding without that simulator.
Finished <<< dolly_ignition [0.83s]
Finished <<< dolly_gazebo [3.69s]
Starting >>> dolly
Finished <<< dolly [1.47s]
Summary: 7 packages finished [17.1s]
1 package had stderr output: dolly_ignition
```

Task 2: Follow me robot

Terminal 1

```
user:~/launch_ws$ cd src/dolly/dolly_gazebo/launch
user:~/launch_ws/src/dolly/dolly_gazebo/launch$ touch demo.launch.py
user:~/launch_ws/src/dolly/dolly_gazebo/launch$ cd ~/launch_ws
```

Open the code editor. Download the demo.launch at politemall then Copy and paste the launch file to the demo.launch.py

line 1

```
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.actions import IncludeLaunchDescription
from launch.conditions import IfCondition
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node
```

line 10



 demo.launch

 talk_listen.launch

 Lab 6 ROS launch files

Copy and paste the remaining code to the demo.launch.py. Save the file.

line 12

```
def generate_launch_description():

    pkg_gazebo_ros = get_package_share_directory('gazebo_ros')
    pkg_dolly_gazebo = get_package_share_directory('dolly_gazebo')

    # Gazebo launch
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(pkg_gazebo_ros, 'launch', 'gazebo.launch.py'),
        )
    )
    # RViz
    rviz = Node(
        package='rviz2',
        executable='rviz2',
        arguments=[
            '-d', os.path.join(pkg_dolly_gazebo, 'rviz', 'dolly_gazebo.rviz']],
        condition=IfCondition(LaunchConfiguration('rviz'))
    )
    return LaunchDescription([
        DeclareLaunchArgument(
            'world',
            default_value=[os.path.join(
                pkg_dolly_gazebo, 'worlds', 'dolly_empty.world'), ""],
            description='SDF world file'),
        DeclareLaunchArgument('rviz', default_value='true',
            description='Open RViz.'),
        gazebo,
        rviz
    ])
```

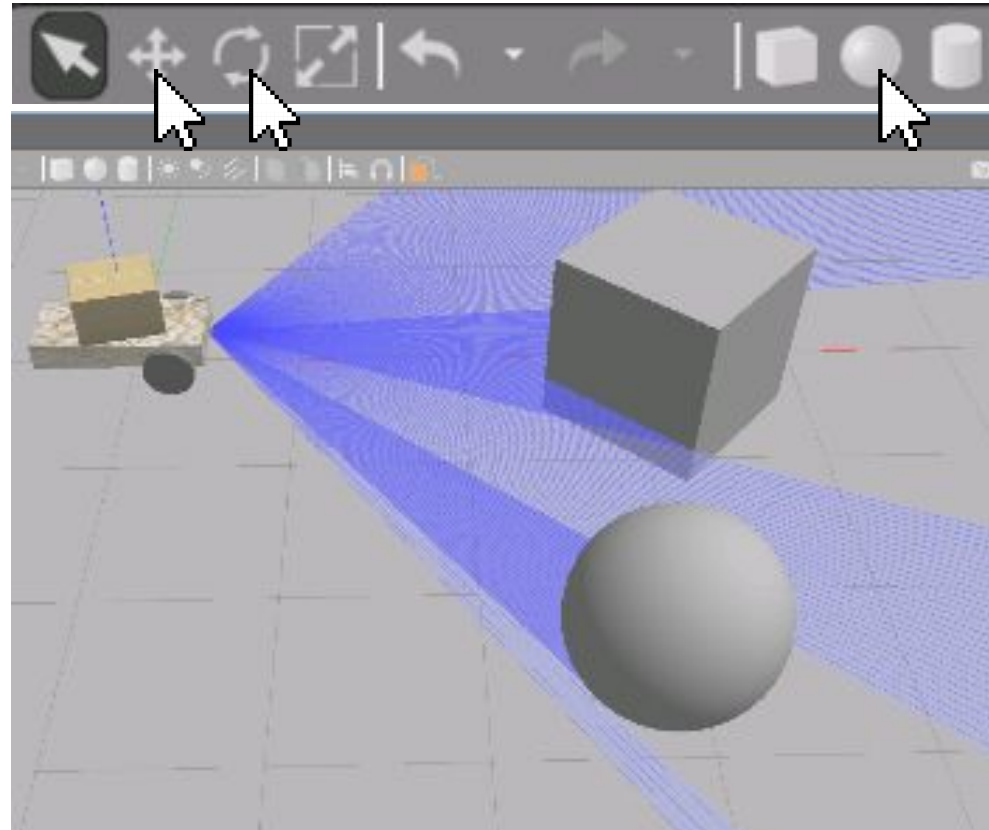
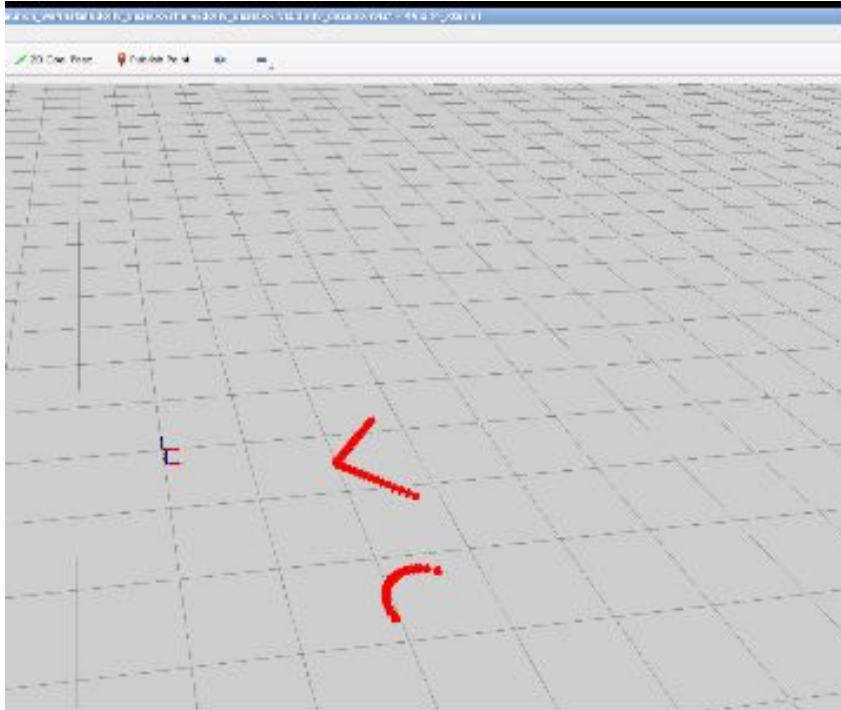

Task 2: Follow me robot

Terminal 1

Launch the demo.launch.py. It will launch the Rviz and gazebo. Add in the objects and rotate such that it is similar to the following figures. Observe the laser scan reflections.

```
user:~/launch_ws/src/dolly/dolly_gazebo/launch$ cd ~/launch_ws  
user:~/launch_ws$ colcon build  
user:~/launch_ws$ source install/setup.bash  
user:~/launch_ws$ ros2 launch dolly_gazebo demo.launch.py world:=dolly_empty.world
```

RViz



Task 2: Follow me robot

Terminal 2

Run the teleop program with remap function. The previous lab example is provided as follows. You may want to open the 3rd Terminal and run **ros2 topic list** to find out what is the name of the old and new topic for remapping. Make sure that you remember to source the setup.bash and rerun ros2 run teleop

```
ros2 run teleop_cpp_ros2 teleop -ros args -remap /cmd_vel:=/turtle1/cmd_vel
```

```
user:~/ $ cd launch_ws
user:~/launch_ws$ ros2 run teleop_cpp_ros2 teleop -ros args -remap ???????????

user:~/launch_ws$ source install/setup.bash
user:~/launch_ws$ ros2 run teleop_cpp_ros2 teleop -ros args -remap ???????????
```

Answer: user:~/launch_ws\$ ros2 run teleop_cpp_ros2 teleop -ros -ros args -remap /cmd_vel:=/dolly/cmd_vel

Task 2: Follow me robot

Open code editor and edit the `launch_ws/src/dolly/dolly_gazebo/launch/demo.launch.py` file.

Add in the highlighted follow me node to the launch file. The follow node will read in the laser scan and generate `/cmd_vel` to control the robot in gazebo simulator. Notice, how remapping is done in the launch file.

Save the file.

```
follow = Node(  
    package='dolly_follow',  
    executable='dolly_follow',  
    output='screen',  
    remappings=[  
        ('cmd_vel', '/dolly/cmd_vel'),  
        ('laser_scan', '/dolly/laser_scan')  
    ]  
)  
  
return LaunchDescription([  
    DeclareLaunchArgument(  
        'world',  
        default_value=[os.path.join(  
            pkg_dolly_gazebo, 'worlds', 'dolly_empty.world'), ""],  
        description='SDF world file'),  
    DeclareLaunchArgument('rviz', default_value='true',  
        description='Open RViz.'),  
    gazebo,  
    follow,  
    rviz  
)
```

Task 2: Follow me robot

Terminal 2

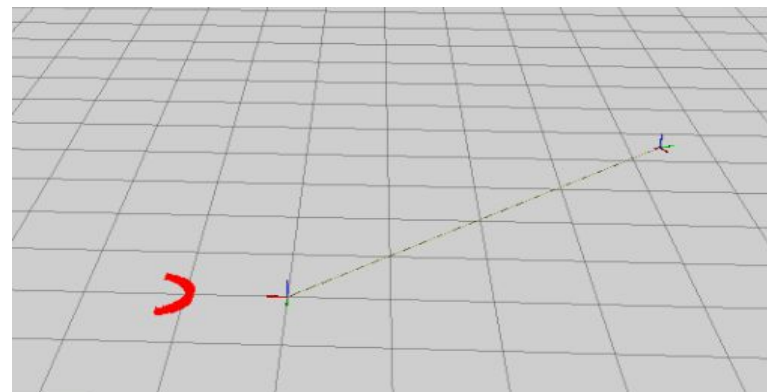
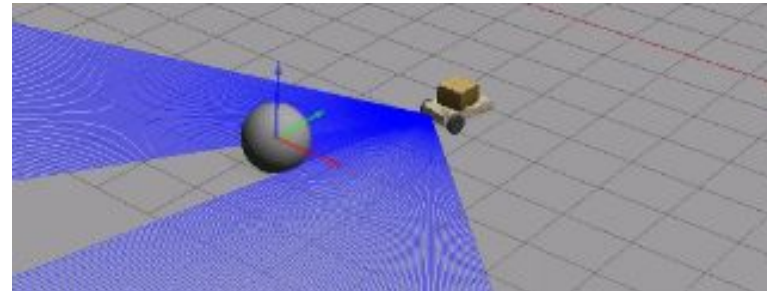
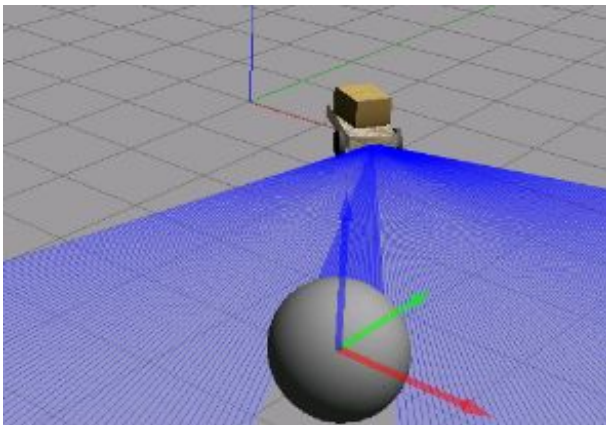
Ctrl+C to stop the teleop program. The robot will be control by the follow node instead of the keyboard.

Terminal 1

Ctrl+C to stop the previous launch file. Build the workspace and launch the new launch file. Add in the object as shown and move the object around to allow the robot to follow.

Observe the laser scan in Rviz.

```
user:~/launch_ws$ colcon build
user:~/launch_ws$ ros2 launch dolly_gazebo demo.launch.py world:=dolly_empty.world
```



Demo and Q&A

Q1 Can you run a ros node without launch file ?

Q2 What is the purpose of using a launch file ?

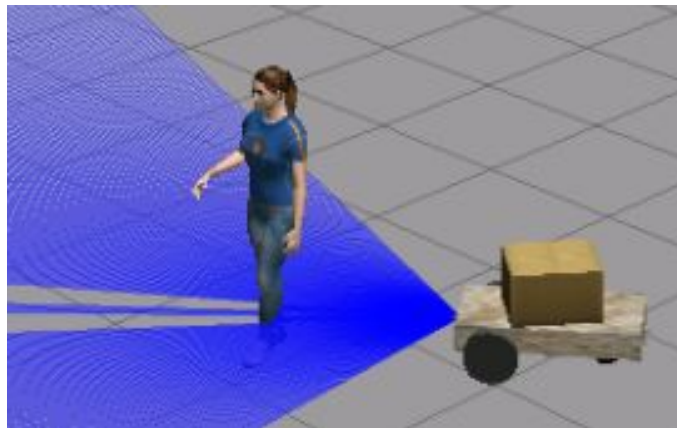
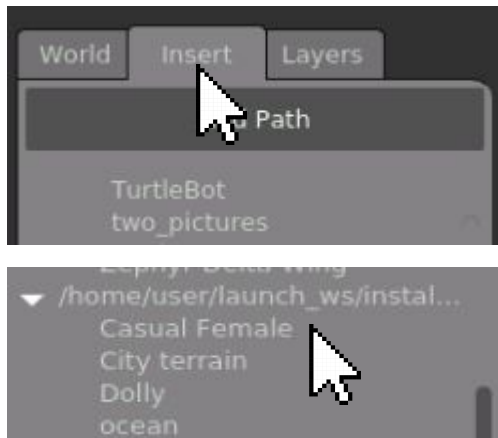
Q3 What is the different between Gazebo and Rviz ?

Q4 What is the function of dolly_empty.world ?

<https://nlamprian.me/blog/software/ros/2019/10/06/gazebo-virtual-worlds/>

Q5 Demo the **follow me** robot to your instructor.

You can try to add in any other objects to the world. Try it !



End of briefing for Lab session